Deep Learning Project

Implementation Analysis Report

Course: Deep Learning & Neural Networks - Fall 2026

Project Domain: Multi-Modal Restaurant Quality Prediction

Approach: Web Scraping of Restaurant Reviews, Images, and Tabular Data

Comprehensive evaluation of project implementation against course requirements and grading rubric.

Generated: May 13, 2026

Table of Contents

1. Executive Summary

This report provides a comprehensive analysis of the Deep Learning project implementation for the Fall 2026 course at Helwan National University. The project aims to build a multi-modal restaurant quality prediction system using web scraping from restaurant review websites, combined with five different deep learning architectures: a Deep Neural Network (DNN), a Convolutional Neural Network (CNN) with transfer learning, an LSTM for text sentiment analysis, a Transformer model, and a Multi-Modal ensemble combining image and tabular features.

After thorough analysis of all seven notebooks, the dataset files, the grading rubric, and the project requirements, the implementation demonstrates a reasonable structural foundation with clear notebook organization and an appropriate selection of model architectures. However, several **critical issues** were identified that significantly impact the validity and quality of the project. The most severe issue is a **data leakage problem** in the DNN and Multi-Modal models, where the target variable (restaurant rating) is included as an input feature. Additionally, the dataset is synthetically generated rather than actually scraped from the web, which contradicts the stated project approach, and the dataset is extremely small (only 150 samples) with very limited text variety (only 15 review templates).

The LSTM model achieving 100% accuracy on the test set is a clear indicator of data memorization rather than genuine learning, and the CNN model performing at only 57% accuracy (barely above random) reveals that the images have no meaningful correlation with the prediction task. There is also a complete absence of comparative analysis between models, no hyperparameter tuning, and no cross-validation, all of which are important components expected in a university deep learning project.

Grading Criterion	Max Marks	Estimated Score	Status
Data & Problem Understanding	3	1.5 - 2	Partial
Modeling & Architecture Design	5	2.5 - 3	Needs Work
Implementation & Technical Quality	2	0.5 - 1	Critical Issues
Experimental & Comparative Analysis	2	0 - 0.5	Missing
Teamwork & Individual Contribution	2	1 - 1.5	Adequate
Evaluation Metrics & Validation Strategy	1	0.3 - 0.5	Weak
Individual Understanding	5	Depends on Viva	Pending
TOTAL	**20**	**5.8 - 9.0** (before viva)	**At Risk**

Table 1: Estimated Grading Score Summary

2. Grading Rubric Overview

The course provides a single-page cover sheet that serves as the official grading rubric for the Deep Learning project. The total project is worth 20 marks, distributed across seven criteria. Understanding each criterion in detail is essential for ensuring the implementation meets the expected standards. Below is a detailed breakdown of what each criterion demands and how it is likely assessed by the instructor.

Criterion	Marks	What It Requires
Data & Problem Understanding	3	Demonstrate deep understanding of the data domain, data characteristics, feature relationships, and the problem being solved. Show that you know why each feature matters and how the data supports the chosen modeling approach.
Modeling & Architecture Design	5	Design architectures with justification. Explain why specific layer types, sizes, activations, and hyperparameters were chosen. Show architectural diagrams. Demonstrate innovation and understanding of each model type.
Implementation & Technical Quality	2	Clean, bug-free, well-documented code. Correct use of APIs. No data leakage. Proper preprocessing pipeline. Professional code structure.
Experimental & Comparative Analysis	2	Compare models against each other. Perform ablation studies. Try different hyperparameters and report results. Show learning curves and analyze training behavior.
Teamwork & Individual Contribution	2	Each team member must have a distinct, meaningful contribution. Clear assignment of responsibilities. Evidence of collaboration.
Evaluation Metrics & Validation Strategy	1	Appropriate metric selection for each task. Proper train/val/test splitting. Cross-validation where applicable. Statistical significance tests.
Individual Understanding	5	Each team member must demonstrate deep understanding of their assigned model during the presentation/viva. Ability to answer questions about architecture, training, results.

Table 2: Detailed Grading Rubric Analysis

3. Project Requirements Analysis

Based on the cover sheet and the nature of the Deep Learning course project, the following requirements can be inferred. The project requires each team to choose a domain, build a custom dataset (not from Kaggle or ready-made sources), implement five different deep learning models corresponding to five team members, and demonstrate understanding through implementation, analysis, and presentation. The chosen domain for this project is restaurant quality prediction using multi-modal data (tabular, text, and images) scraped from restaurant review websites.

3.1 Explicit Requirements

- Choose a project domain and problem statement
- Build a custom dataset (not from Kaggle, not ready-made)
- Implement at least five different deep learning models
- Each team member is responsible for one model
- Use multi-modal data (at least two data types)
- Utilize pre-trained models where applicable
- Demonstrate innovation beyond basic model implementations
- Focus on understanding, not just accuracy

3.2 Implicit Requirements (from Grading Rubric)

- Comprehensive data exploration and visualization
- Justified architecture decisions with explanations
- Hyperparameter tuning and experimentation
- Comparative analysis between different models
- Ablation studies to understand component contributions
- Proper cross-validation and evaluation strategies
- Confusion matrices and detailed classification reports
- Each team member must deeply understand their model

4. Implementation Deep-Dive

This section provides a detailed analysis of each notebook in the project, examining the code quality, correctness, completeness, and alignment with the grading criteria. Each notebook is evaluated independently, followed by a cross-cutting analysis of systemic issues.

4.1 Data Scraping & Generation (01_Data_Scraping.ipynb)

> **[CRITICAL]** The notebook title says "Data Scraping" but the implementation does NOT actually scrape any website. All data is synthetically generated using random functions and template lists.

The first notebook is labeled as "Data Generation" in its markdown heading but the file name says "Data_Scraping." This discrepancy itself is a concern. The original project plan was to perform web scraping from a restaurant review website, but due to anti-bot protection (as mentioned in the README), the team pivoted to generating synthetic data. While the README acknowledges this limitation, it represents a significant deviation from the stated project approach.

Specific Issues:

- **No actual web scraping:** The notebook generates restaurant names from a hardcoded list of 20 names, cuisines from 10 options, neighborhoods from 7 options, and prices from 4 tiers. Ratings are

generated using a Beta distribution, and review counts are computed as a simple linear function of the rating plus noise.

- **Only 15 unique review templates:** There are exactly 5 positive reviews, 5 neutral reviews, and 5 negative reviews in the entire dataset of 150 restaurants. Each restaurant gets one review chosen deterministically based on its rating bin. This means many restaurants share identical review text, making the text classification task trivially easy and unrepresentative of real-world data.
- **Only 19 unique images:** The images are downloaded from 19 Unsplash URLs and rotated across the 150 restaurants. This means roughly every 8th restaurant shares the same image, and the images have no correlation with restaurant quality. When downloads fail (15 out of 150), failed images are replaced by copying a random successful image.
- **Deterministic generation:** Using fixed random seeds and modular indexing (e.g., cuisine_type = cuisines[i % 10]) creates strong artificial patterns that do not exist in real data, which can lead to misleadingly high model performance on this synthetic data.
- **Only 150 samples:** For deep learning, 150 samples is extremely small. Even simple models risk overfitting. For image classification with MobileNetV2 (which has millions of parameters), 120 training images is far too few for meaningful learning.

> **[WARNING]** The dataset size and variety are insufficient for meaningful deep learning experiments. A real scraping effort (even with a smaller website or a different approach) would be significantly more valuable for the project.

4.2 Data Preprocessing (02_Data_Preprocessing.ipynb)

> **[CRITICAL]** The preprocessing pipeline includes "rating_scaled" as a tabular feature, which is the normalized version of the target variable. This creates a data leakage problem in the DNN and Multi-Modal models.

The preprocessing notebook is reasonably structured, handling three data modalities (tabular, text, images) with appropriate encoding and normalization techniques. However, there is a fundamental flaw in the feature selection for the tabular data pipeline.

Specific Issues:

- **Data Leakage in Tabular Features:** The tabular features include "price_encoded", "cuisine_encoded", "neighborhood_encoded", "rating_scaled", and "review_count_scaled". The DNN model (Notebook 03) and the Multi-Modal model (Notebook 07) use these features to predict "rating" as the target. Since "rating_scaled" is just a StandardScaler-transformed version of "rating", the model is essentially learning to predict the target from the target itself. This invalidates the results of both the DNN and Multi-Modal models.
- **StandardScaler applied to rating:** Using StandardScaler on the rating (which is the target variable) and then including it as an input feature is a methodological error. The model can trivially recover the original rating from the scaled version, making the results meaningless.
- **No validation set:** The data is split only into train (80%) and test (20%). During model training, a validation split is taken from the training data, but there is no separate, held-out validation set

created during preprocessing. This is acceptable but not ideal for proper hyperparameter tuning.

- **Missing text preprocessing:** There is no text cleaning (lowercasing, removing punctuation, removing stop words) before tokenization. While tokenizers can handle some of this, explicit preprocessing could improve model performance on real-world text data.

4.3 Model 1: DNN (03_Model_1_DNN.ipynb)

> **[CRITICAL]** This model uses "rating_scaled" as an input feature while predicting "rating" as the output. The MAE of 0.73 on the test set is artificially low because the model has direct access to the target.

The DNN model is a straightforward feedforward network with three hidden layers (64, 32, 16 neurons), Dropout regularization (0.3, 0.2), and a linear output for regression. The architecture is basic but appropriate for a tabular data task. However, the data leakage issue completely undermines the results.

What Needs Improvement:

- **Remove "rating_scaled" from input features** - this is the target variable
- Add architecture justification: Why 64-32-16? Why these dropout rates? What alternatives were tried?
- Add hyperparameter experiments: different layer sizes, dropout rates, learning rates
- Include residual analysis: plot predicted vs actual ratings, error distribution
- Add a baseline comparison: how does the DNN compare to a simple mean prediction or linear regression?

4.4 Model 2: CNN (04_Model_2_CNN.ipynb)

> **[WARNING]** The CNN achieves only 56.67% accuracy on binary classification, barely above random guessing (50%). This is expected since the images have no meaningful correlation with restaurant quality tiers.

The CNN model uses MobileNetV2 pre-trained on ImageNet as a feature extractor with frozen layers, adding a GlobalAveragePooling2D layer, a Dense(128) layer with Dropout(0.5), and a sigmoid output. The approach is structurally sound and demonstrates transfer learning, which is a valid deep learning technique. However, the fundamental problem is that the images used in this project are generic food photos from Unsplash that have no relationship to whether a restaurant is "high-tier" or "low-tier."

What Needs Improvement:

- Fine-tune the top layers of MobileNetV2 instead of keeping all layers frozen
- Add data augmentation (rotation, flipping, color jittering) to increase effective dataset size
- Show confusion matrix analysis (imported but never used in the notebook)
- Experiment with different learning rates and unfreezing strategies
- Consider whether image-based prediction of restaurant quality is even a valid task with this data

4.5 Model 3: LSTM (05_Model_3_LSTM.ipynb)

> **[CRITICAL]** The LSTM achieves 100% accuracy on the test set, which is a clear sign of data memorization / overfitting, not genuine learning. With only 15 unique review templates and a 30-sample test set, the model can easily memorize all patterns.

The LSTM model uses a Bidirectional architecture with two LSTM layers (64 and 32 units), embedding dimension of 64, and dropout of 0.5 and 0.3. The architecture design is reasonable and demonstrates understanding of RNN concepts. However, the perfect accuracy is misleading and would raise serious concerns during evaluation.

The reason for the 100% accuracy is straightforward: the dataset has only 15 distinct review texts (5 positive, 5 neutral, 5 negative), and each review is deterministically assigned to a sentiment class based on the restaurant rating. The test set of 30 samples contains at most a few unique review texts per class, making it trivially classifiable. A real test of the model would require unseen review texts that were not in the training set, which is impossible with this dataset structure.

What Needs Improvement:

- Use a larger, more diverse dataset with hundreds of unique reviews
- Ensure test set contains truly unseen review texts, not just unseen restaurant entries
- Add confusion matrix visualization and error analysis
- Experiment with different embedding dimensions, LSTM units, and dropout rates
- Add a baseline comparison: TF-IDF + Logistic Regression, or pre-trained word embeddings

4.6 Model 4: Transformer (06_Model_4_Transformer.ipynb)

> **[WARNING]** The Transformer achieves 93.3% accuracy but completely fails on the "Negative" class (0% recall). This means it never correctly identifies a negative sentiment review, which is a significant weakness.

The Transformer model uses TensorFlow native MultiHeadAttention with 2 heads, embedding dimension of 64, and feed-forward dimension of 64. The choice to use TensorFlow native layers instead of HuggingFace transformers is reasonable and demonstrates understanding of the core attention mechanism. However, there are several issues with the implementation and results.

What Needs Improvement:

- **Missing positional encoding:** The Transformer architecture requires positional information to understand word order. Without positional encoding, the model treats the input as a bag of words, which defeats the purpose of using a Transformer over simpler architectures.
- **Class imbalance handling:** The dataset has only 7 negative samples out of 150, compared to 71 positive and 72 neutral. No techniques (class weights, oversampling, SMOTE) are used to address this.
- **Single Transformer block:** Only one encoder block is used. Experiment with 2-3 blocks and analyze the effect on performance.

- **No comparative analysis:** Compare the Transformer results directly with the LSTM results on the same data, discussing advantages and disadvantages.
- Add learning rate scheduling (warmup + decay) which is standard for Transformer training
- Show attention weights visualization to demonstrate that the model is learning meaningful patterns

4.7 Model 5: Multi-Modal Ensemble (07_Model_5_MultiModal.ipynb)

> **[CRITICAL]** Like the DNN, this model uses tabular features that include "rating_scaled" (the target), making its results invalid. Additionally, the image branch provides no useful signal since images have no correlation with ratings.

The multi-modal model is the most architecturally interesting component of the project. It combines a CNN branch (MobileNetV2 feature extractor) with a DNN branch (Dense 32, Dropout 0.3) through concatenation, followed by Dense(64), Dropout(0.5), Dense(32), and a linear output. This is a legitimate multi-modal fusion approach that demonstrates innovation. However, the data leakage and the lack of useful image features undermine the results.

What Needs Improvement:

- **Fix data leakage:** Remove "rating_scaled" from tabular features immediately
- **Add text modality:** A true multi-modal model should incorporate all three data types (tabular + text + images), not just two.
- **Ablation study:** Show results with (a) tabular only, (b) images only, (c) combined. This demonstrates the value of multi-modal fusion.
- **Late fusion vs early fusion:** Try different fusion strategies and compare results.
- **Gated fusion:** Instead of simple concatenation, implement attention-based or gated fusion that learns to weight each modality differently.
- Compare multi-modal results against individual models to quantify the improvement

5. Critical Issues Found

This section summarizes the most severe problems found in the implementation, ranked by their impact on the project grade and the validity of the results. Each issue is classified by severity: Critical (must fix before submission), Major (strongly recommended to fix), and Minor (nice to have but not grade-threatening).

#	Issue	Severity	Affected Models	Impact
1	Data Leakage: rating_scaled used as both input and target	CRITICAL	DNN, Multi-Modal	Invalidates results completely
2	Synthetic data, not actually scraped from web	CRITICAL	All	Contradicts project approach

3	Only 15 unique review templates for 150 restaurants	CRITICAL	LSTM, Transformer	Inflated accuracy (100% LSTM)
4	No comparative analysis between models	MAJOR	All	2 marks lost (Experimental Analysis)
5	No hyperparameter tuning or experimentation	MAJOR	All	Weak Modeling score
6	Missing positional encoding in Transformer	MAJOR	Transformer	Fundamental architectural flaw
7	Only 19 unique images recycled for 150 restaurants	MAJOR	CNN, Multi-Modal	No image-task correlation
8	No confusion matrices visualized	MINOR	CNN, LSTM, Transformer	Missing evaluation depth
9	No cross-validation	MINOR	All	Weaker validation
10	No data augmentation for images	MINOR	CNN, Multi-Modal	Underfitting risk

Table 3: Critical Issues Ranked by Severity

6. Grading Criteria Assessment

6.1 Data & Problem Understanding (3 marks)

The project demonstrates some understanding of the restaurant quality prediction problem and the multi-modal approach. The README and markdown cells explain the domain and data types adequately. However, the understanding is shallow in several key areas. There is no exploratory data analysis (EDA) notebook or section that examines feature distributions, correlations, or class balance. The relationship between the data modalities is not analyzed. The team acknowledges that real scraping failed but does not discuss the implications of using synthetic data on model validity. A deeper analysis of why certain features were chosen, what their distributions look like, and how they interact would significantly improve this score.

Estimated Score: 1.5 - 2 / 3

6.2 Modeling & Architecture Design (5 marks)

This is the highest-weighted criterion (5 marks) and the current implementation is weak here. The architectures are presented without justification. Why does the DNN use 64-32-16 neurons? Why is the embedding dimension 64? Why 2 attention heads in the Transformer? Why is the LSTM bidirectional? These decisions should be explained with reasoning, not just implemented. There are no architectural diagrams. No alternative architectures were explored. The multi-modal model is the most innovative component, but even it lacks ablation studies showing the contribution of each modality. The

Transformer is missing positional encoding, which is a fundamental component that should not be omitted. No learning rate scheduling or training strategies beyond basic early stopping are used.

Estimated Score: 2.5 - 3 / 5

6.3 Implementation & Technical Quality (2 marks)

The data leakage issue (using rating_scaled as an input while predicting rating) is a serious implementation flaw that would likely result in a very low score on this criterion. In addition, the notebooks import confusion_matrix but never use it. The code structure is clean but lacks functions for reusability. The notebooks are linear scripts rather than modular code. The fact that the LSTM achieves 100% accuracy and nobody questioned this result suggests insufficient critical review of the outputs. A technically sound implementation should not produce obviously flawed results without investigation.

Estimated Score: 0.5 - 1 / 2

6.4 Experimental & Comparative Analysis (2 marks)

This criterion is essentially unaddressed in the current implementation. There is no comparison between models, no ablation studies, no hyperparameter experiments, and no analysis of training curves beyond simple loss/accuracy plots. The models are trained independently with no discussion of relative strengths and weaknesses. This is the easiest section to improve, as it primarily requires adding analysis rather than changing the models themselves. A comparison table showing all five models side-by-side with their metrics, training times, and trade-offs would be a strong starting point.

Estimated Score: 0 - 0.5 / 2

6.5 Teamwork & Individual Contribution (2 marks)

The notebook structure naturally lends itself to team division, with each model in a separate notebook. The README includes a "Team Member Responsibilities" section. However, there is no explicit documentation of which team member was responsible for which notebook, and no git history or contribution log is provided. The cover sheet requires team member names and individual grades, suggesting that the instructor evaluates individual contributions carefully. More explicit documentation of task assignments and contributions would strengthen this criterion.

Estimated Score: 1 - 1.5 / 2

6.6 Evaluation Metrics & Validation Strategy (1 mark)

The project uses appropriate metrics for each task (MSE/MAE for regression, accuracy for classification) but does not go beyond basic evaluation. There is no cross-validation, no confidence intervals, no statistical significance tests, and no analysis of whether the results are meaningful. The validation strategy relies solely on a single train/test split with an internal validation split during training. For a dataset this small, cross-validation would be particularly important to assess the stability of the results.

Estimated Score: 0.3 - 0.5 / 1

6.7 Individual Understanding (5 marks)

This criterion is assessed during the presentation/viva and cannot be evaluated from the code alone. However, the quality of the notebooks provides some indication. The markdown explanations are brief and generic, which suggests that team members may struggle to answer deep questions about their models. Each team member should be able to explain: (a) why they chose their specific architecture, (b) what each layer does and why it is necessary, (c) what hyperparameters they experimented with and why, (d) what their results mean and whether they are reasonable, and (e) how their model compares to others in the team.

> **Estimated Score: Depends entirely on viva performance**

7. Recommendations & Action Items

The following recommendations are organized by priority. The "Must Fix" items are critical and should be addressed before submission to avoid losing significant marks. The "Should Fix" items would substantially improve the grade. The "Nice to Have" items would polish the project further.

7.1 Must Fix (Before Submission)

Fix 1: Remove Data Leakage

In the preprocessing notebook (02_Data_Preprocessing.ipynb), remove "rating_scaled" from the tabular features. The tabular features should only include: price_encoded, cuisine_encoded, neighborhood_encoded, and review_count_scaled. Then retrain the DNN and Multi-Modal models with the corrected features. The DNN should now predict rating from the remaining features (price, cuisine, neighborhood, review count) without having access to the answer.

Fix 2: Add Positional Encoding to Transformer

Implement a positional encoding layer in the Transformer model. The standard sinusoidal positional encoding is straightforward to implement and is essential for the Transformer to understand word order. Without it, the model treats each word position identically, making it no better than a bag-of-words approach. This is a fundamental component of the Transformer architecture that demonstrates understanding of the model design.

Fix 3: Add a Comparative Analysis Notebook

Create an additional notebook (e.g., 08_Model_Comparison.ipynb) that loads all five trained models and compares them systematically. Include a comparison table with metrics, training time, parameter counts, and a discussion of trade-offs. This directly addresses the "Experimental & Comparative Analysis" criterion worth 2 marks, which is currently completely missing.

7.2 Should Fix (Significant Grade Improvement)

Fix 4: Expand the Dataset

If actual web scraping is not feasible, at minimum generate a much larger and more diverse synthetic dataset. Use 500-1000 restaurants, generate unique review text for each restaurant (using templates with random variation, or using a language model to generate diverse reviews), and use a wider variety of images. The goal is to ensure that test samples are genuinely unseen, not just different indices into the same small pool of templates.

Fix 5: Add Hyperparameter Experiments

For each model, experiment with at least 2-3 different hyperparameter configurations. For example: try different learning rates (0.01, 0.001, 0.0001), different layer sizes, different dropout rates, or different batch sizes. Document the results in a table and discuss which configuration worked best and why. This demonstrates understanding of the training process and the effect of hyperparameters.

Fix 6: Add Architecture Justifications

For each model, add detailed markdown cells explaining the architectural decisions. Why this number of layers? Why these activation functions? Why this optimizer? Why this learning rate? What alternatives were considered? This directly addresses the "Modeling & Architecture Design" criterion worth 5 marks.

Fix 7: Add Confusion Matrices and Error Analysis

The CNN, LSTM, and Transformer notebooks import confusion_matrix from sklearn but never call it. Add confusion matrix visualizations and detailed error analysis for all classification models. Discuss which classes are being confused and why. For regression models, add predicted-vs-actual scatter plots and residual analysis.

7.3 Nice to Have (Polish)

- Add a dedicated EDA notebook with feature distributions, correlations, and class balance analysis
- Implement cross-validation (k-fold) for more robust evaluation
- Add attention weight visualization for the Transformer model
- Implement data augmentation for the CNN (rotation, flipping, color jittering)
- Add learning rate scheduling for the Transformer (warmup + decay)
- Try late fusion vs. early fusion in the Multi-Modal model
- Add a baseline model comparison (e.g., Logistic Regression, Random Forest)
- Add model architecture diagrams using visual tools

8. Overall Verdict

The current implementation provides a reasonable structural foundation for the Deep Learning project. The choice of five distinct model architectures (DNN, CNN, LSTM, Transformer, Multi-Modal) is appropriate and covers a wide range of deep learning techniques. The multi-modal approach combining tabular, text, and image data demonstrates innovation. The notebook organization is clean and follows a

logical sequence from data collection through preprocessing to model training.

However, the project suffers from several critical issues that severely undermine the validity of the results and the overall quality. The data leakage problem in the DNN and Multi-Modal models is the most urgent issue, as it makes the results of these two models completely unreliable. The synthetic dataset with only 15 unique review templates and 19 unique images is too small and too uniform for meaningful deep learning experiments, leading to the misleading 100% LSTM accuracy and the near-random CNN accuracy. The complete absence of comparative analysis, hyperparameter tuning, and architecture justification means that the highest-weighted grading criteria are not adequately addressed.

With the current state of the implementation, the estimated grade (excluding the viva) is in the range of 5.8 to 9.0 out of 20, which is below passing threshold at most universities. The good news is that the most impactful fixes are relatively straightforward: removing the data leakage, adding positional encoding to the Transformer, creating a comparative analysis notebook, and adding architecture justifications and hyperparameter experiments. These changes alone could raise the estimated grade to the 12-15 range, which would be a solid passing grade.

Scenario	Estimated Grade	Description
Current State	5.8 - 9.0 / 20	Critical data leakage, no comparison, no justification, synthetic data
After Must-Fix Items	10 - 12 / 20	Data leakage fixed, positional encoding added, comparison notebook added
After Should-Fix Items	13 - 16 / 20	Expanded dataset, hyperparameter experiments, architecture justifications, error analysis
After All Recommendations	16 - 18 / 20	Cross-validation, attention visualization, data augmentation, EDA, baselines

Table 4: Grade Improvement Roadmap

The key takeaway is that the project has a solid skeleton but needs significant meat on the bones. The most time-efficient path to a good grade is to (1) fix the data leakage immediately, (2) add the positional encoding, (3) create a comprehensive comparison notebook, and (4) add detailed architecture justifications to every model notebook. These four changes address the highest-weighted grading criteria and would dramatically improve the project quality. The dataset expansion and hyperparameter experiments are important but can be addressed in a second pass if time is limited.

This report was generated through comprehensive analysis of all project files, including seven Jupyter notebooks, three CSV data files, the grading rubric PDF, and the project README. The assessments and recommendations are based on the grading criteria provided in the course cover sheet and standard expectations for a university-level Deep Learning project.